

What You Need to Provide (In Order)

Step 1: Create a Cross-Account IAM Role

Do this in your AWS account:

1. Go to **IAM Console** → **Roles** → **Create Role**
2. Select **Another AWS account** as trusted entity
3. Enter our testing account ID: `[YOUR_TESTING_FIRM_ACCOUNT_ID]`
4. Check **Require MFA** under conditions (mandatory)
5. Name the role: `Pentest-Engagement-Role`

Trust Policy :

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::[YOUR_TESTING_FIRM_ACCOUNT_ID]:root"},
    "Action": "sts:AssumeRole",
    "Condition": {
      "Bool": {"aws:MultiFactorAuthPresent": "true"},
      "StringEquals": {"aws:SourceAccount": "[YOUR_ACCOUNT_ID]"}
    }
  }]
}
```

Step 2: Attach Required Permissions

Attach these two policies to the role:

Policy	Type	Why Required
<code>arn:aws:iam::aws:policy/SecurityAudit</code>	AWS Managed	Read-only access to audit security config across all services
<code>Pentest-Exploitation-Policy</code>	Custom Inline	Scoped write permissions for active testing (see below)

Custom Inline Policy Template:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ScopedExploitation",
      "Effect": "Allow",
      "Action": [
        "ec2:RunInstances", "ec2:ModifyInstanceAttribute",
        "iam:PassRole", "sts:AssumeRole",
        "s3:PutObject", "s3:DeleteObject",
        "lambda:InvokeFunction", "secretsmanager:GetSecretValue"
      ],
      "Resource": [
        "arn:aws:ec2:*:[YOUR_ACCOUNT_ID]:instance/*",
        "arn:aws:iam:*:[YOUR_ACCOUNT_ID]:role/Pentest-*",
        "arn:aws:s3:::client-pentest-*"
      ],
      "Condition": {
        "StringEquals": {"aws:ResourceTag/EngagementScope": "Pentest-2026-Q2"},
        "DateLessThan": {"aws:CurrentTime": "2026-06-30T23:59:59Z"}
      }
    },
    {
      "Sid": "BlockDestructiveActions",
      "Effect": "Deny",
      "Action": [
        "ec2:TerminateInstances", "rds:DeleteDBInstance",
        "s3:DeleteBucket", "cloudtrail:StopLogging",
        "organizations:*"
      ],
      "Resource": "*"
    }
  ]
}
```

Step 3: Configure Session Settings

- **Max Session Duration:** Set to **4 hours** (240 minutes) in role settings
- **MFA:** Enforce via trust policy condition (already included above)
- **Tag the role:** Add tag **EngagementScope=Pentest-2026-Q2** for tracking

Step 4: Share the Role ARN Securely

Send us this exact string via approved channel:

arn:aws:iam::[YOUR_ACCOUNT_ID]:role/Pentest-Engagement-Role

✅ **Approved delivery methods:** AWS Secrets Manager, PGP-encrypted email, or your secure client portal

❌ **Never send:** Plain text email, Slack, or unencrypted messages

Scope Definition:

Provide These 5 Items (Example):

1. **Account IDs:** 123456789012, 987654321098
2. **Regions:** us-east-1, eu-west-1
3. **Resource Tags:** Environment=Production, DataClassification=Confidential
4. **VPC CIDRs:** 10.10.0.0/16, 172.31.0.0/16
5. **Out-of-Scope:** RDS-Prod-DB, Salesforce-Integration, DoS Testing

Critical Rules to Confirm:

- ✅ Testing allowed on: EC2, S3, IAM, Lambda, API Gateway (per AWS policy)
- ❌ Testing prohibited on: Route 53 DNS manipulation, DoS/DDoS, S3 bucket takeover

Pre-Engagement Validation

We will run these 3 checks before testing starts:

1. Verify role assumption works

```
aws sts assume-role --role-arn [ROLE_ARN] --role-session-name test
```

2. Confirm SecurityAudit permissions

```
aws iam get-account-authorization-details --max-items 1
```

3. Test resource access

```
aws ec2 describe-instances --filters "Name=tag:EngagementScope,Values=Pentest-2026-Q2"
```

✅ Success = Commands return expected results without **AccessDenied** errors



Emergency Controls (You Retain Full Control)

Action	How to Execute	Response Time
Revoke access	Delete/modify the IAM role trust policy	Immediate
Pause testing	Remove Pentest-Exploitation-Policy from role	< 1 minute
Full lockdown	Enable AWS Organizations SCP denying all testing actions	< 5 minutes

i Why This Setup? > - Uses temporary credentials (not long-term keys) per AWS IAM best practices

- Follows least-privilege principle: read-only baseline + scoped write permissions
- Aligns with AWS penetration testing policy: no prohibited activities, clear boundaries
- Enables full auditability: all actions logged to CloudTrail with assuming principal identity